

# CVE-2026-12003

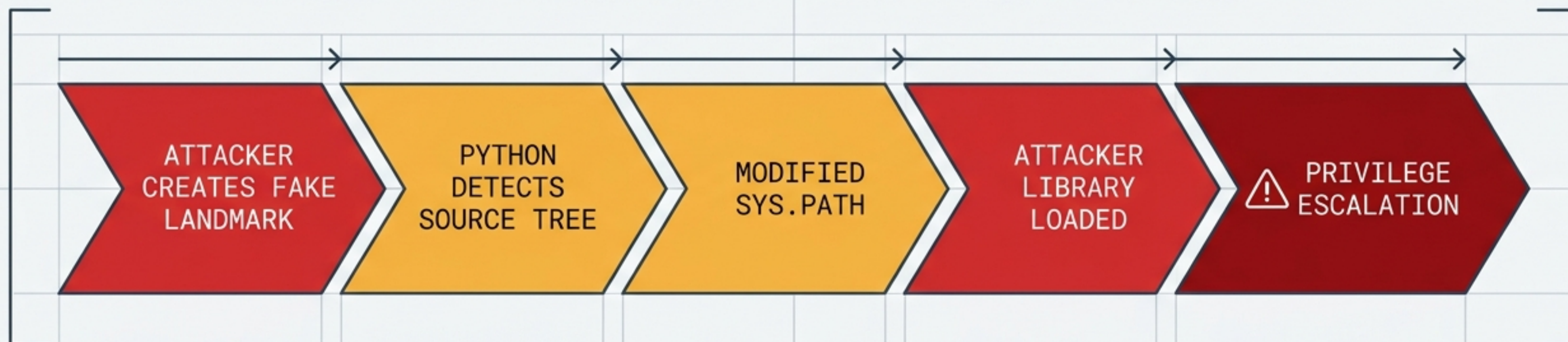
## Intelligence Brief: CPython VPATH Privilege Escalation

| Severity                                                                                         | Vulnerability Type   | Platform Target | CWE Designation |
|--------------------------------------------------------------------------------------------------|----------------------|-----------------|-----------------|
| <br>5.3 Medium | Privilege Escalation | Windows         | CWE-427         |

A flaw in CPython's VPATH landmark detection allows attackers to force the execution of malicious Python libraries by tricking the system into identifying an untrusted directory as its source tree.



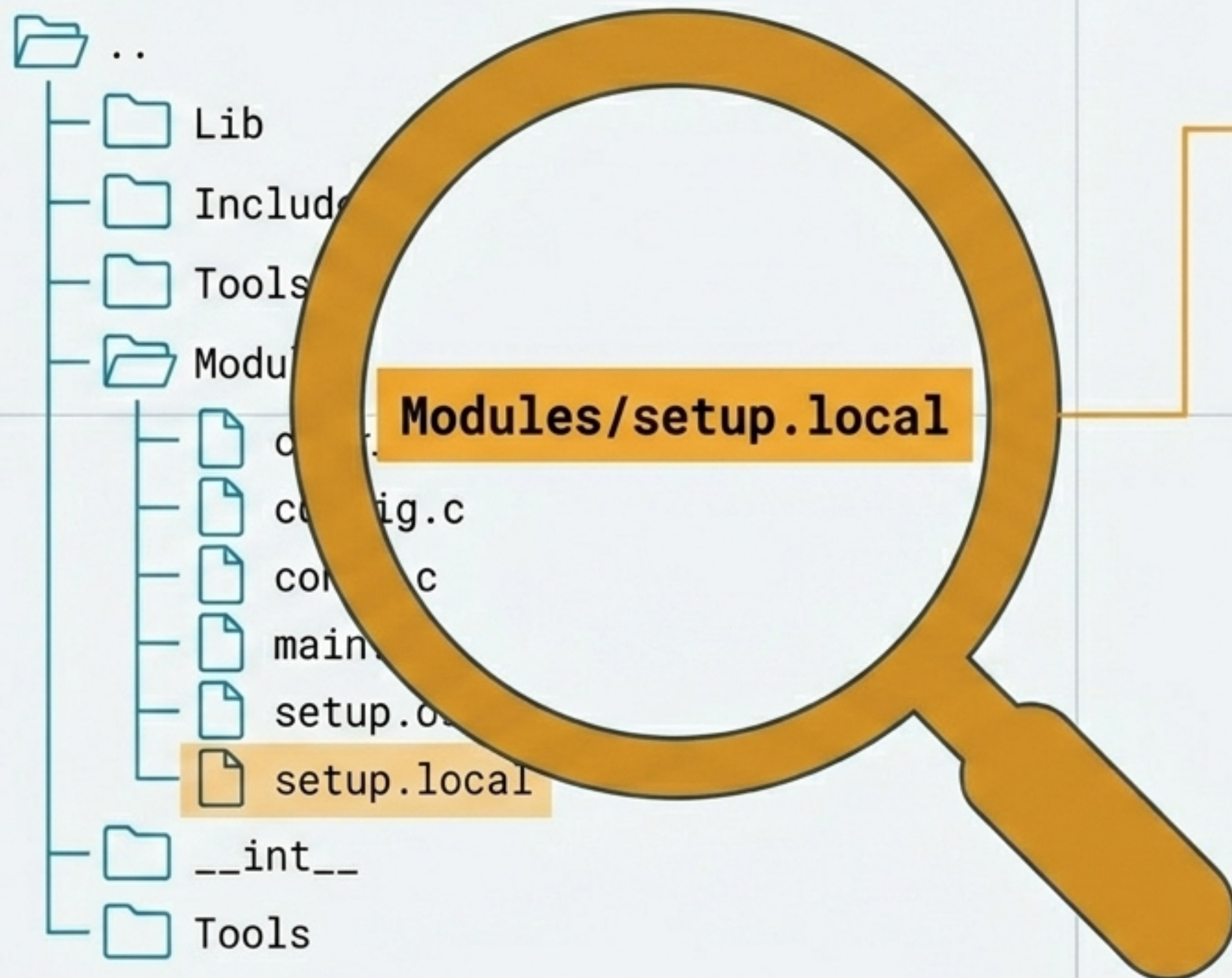
## THE CORE VULNERABILITY MECHANISM



By manipulating the environment's landmark detection, standard execution paths are hijacked outside the intended installation directory.



## TECHNICAL CONTEXT: UNDERSTANDING VPATH



VPATH is a build-time variable used to locate files required during Python development builds.

Python explicitly checks for this landmark file to determine whether it is currently running inside a legitimate source tree.



## Root Cause: The Trust Boundary Violation

Safe Zone

Untrusted Zone

`VPATH = ..\..`

`..\..\Modules\setup.local`

Because VPATH utilizes relative parent directory traversal, the required landmark exists completely outside the protected installation directory, allowing external manipulation.



## The Environmental Factor: Why Windows?

### Standard Windows Security



Modern directories like Program Files are strictly locked down by default OS permissions, preventing unauthorized file creation.

### Legacy Python + C:\ Drive



Legacy Python installations often default to the root drive in paths such as C:\PythonXX.



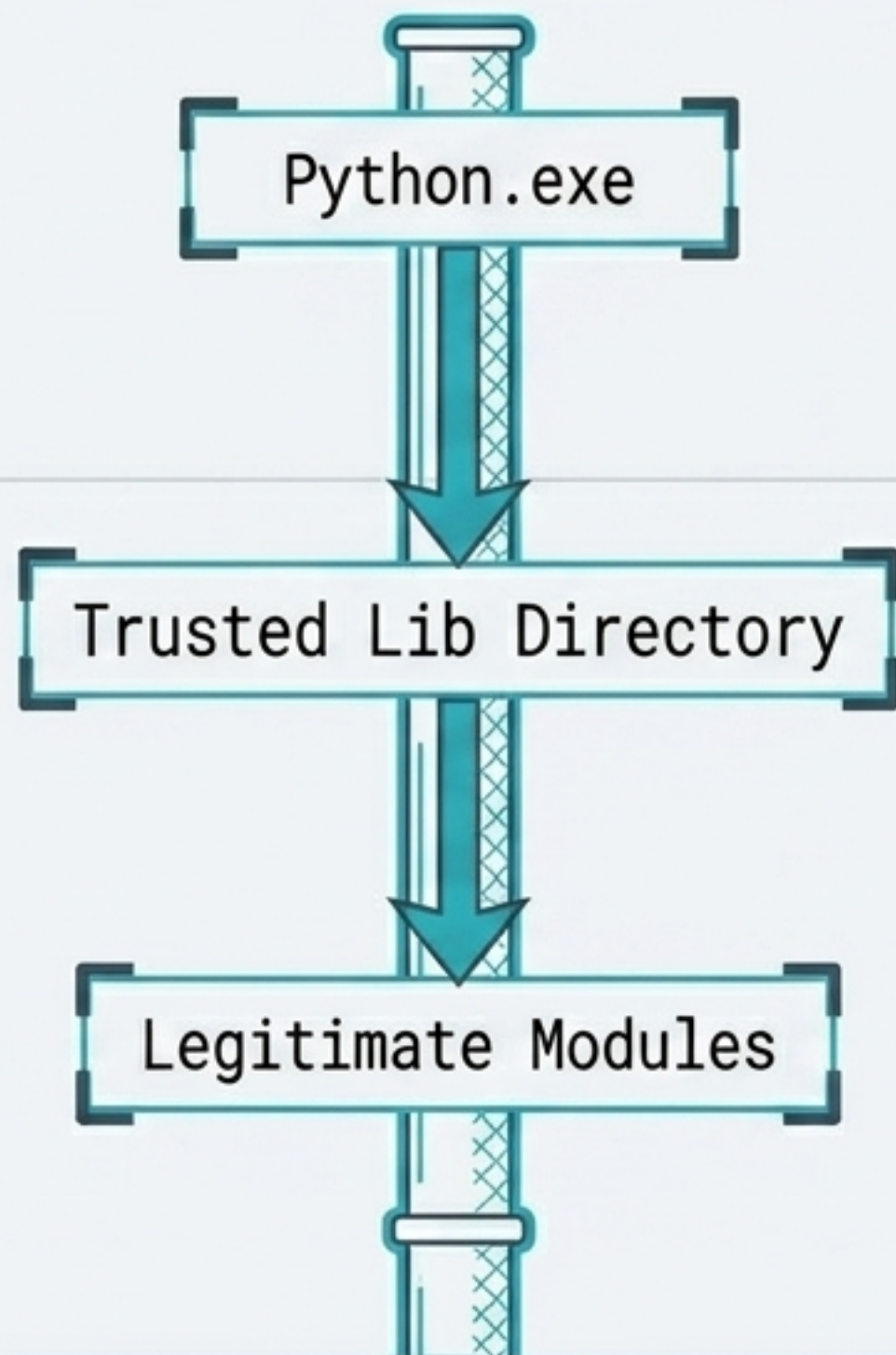
-----> **C:\Modules\**

Because Windows allows standard, low-privilege users to create folders at the root C:\ level, the attacker has the necessary access to stage the malicious files.

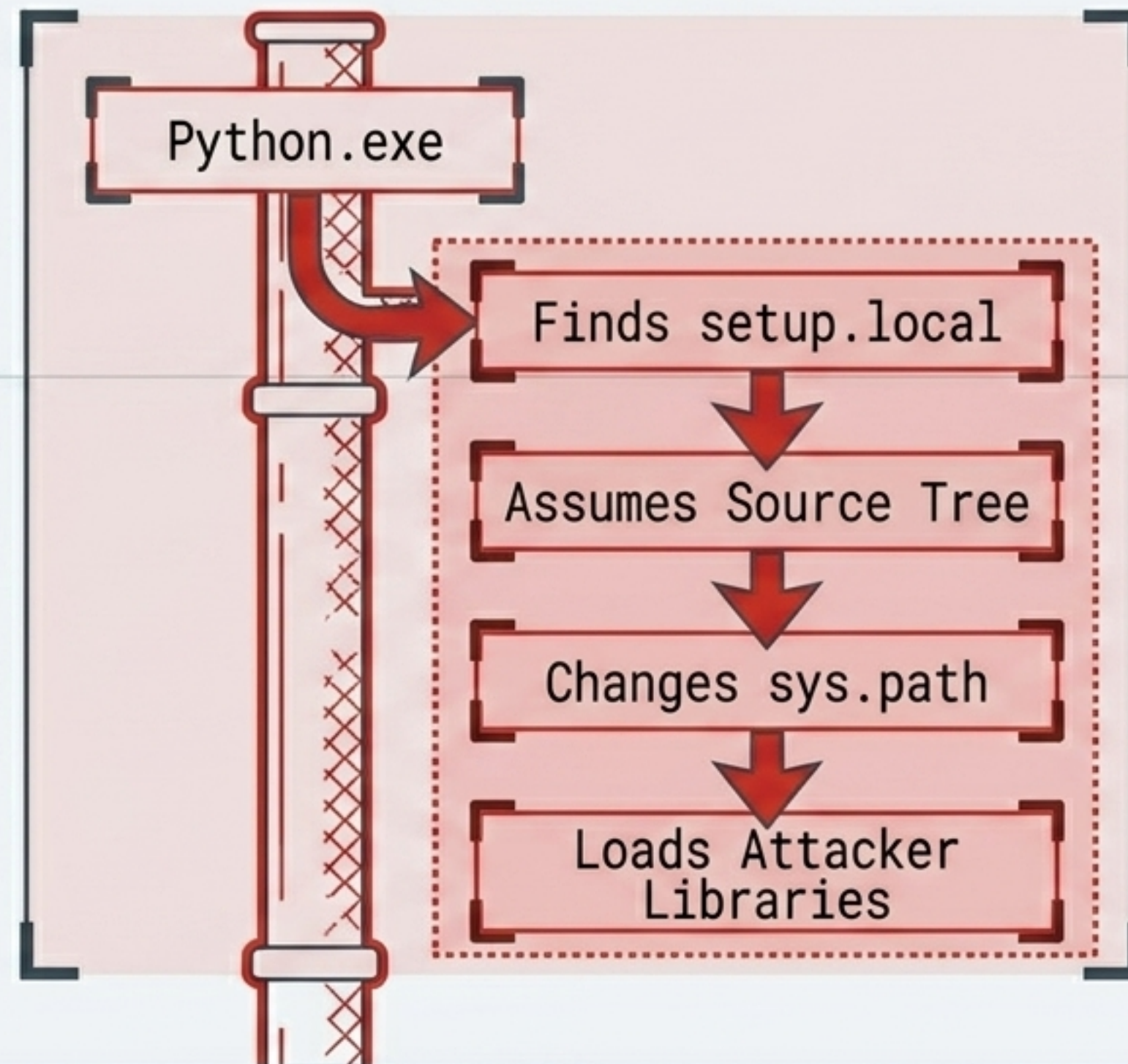


## Architectural Comparison: Execution Paths

### Normal Execution



### Vulnerable Execution

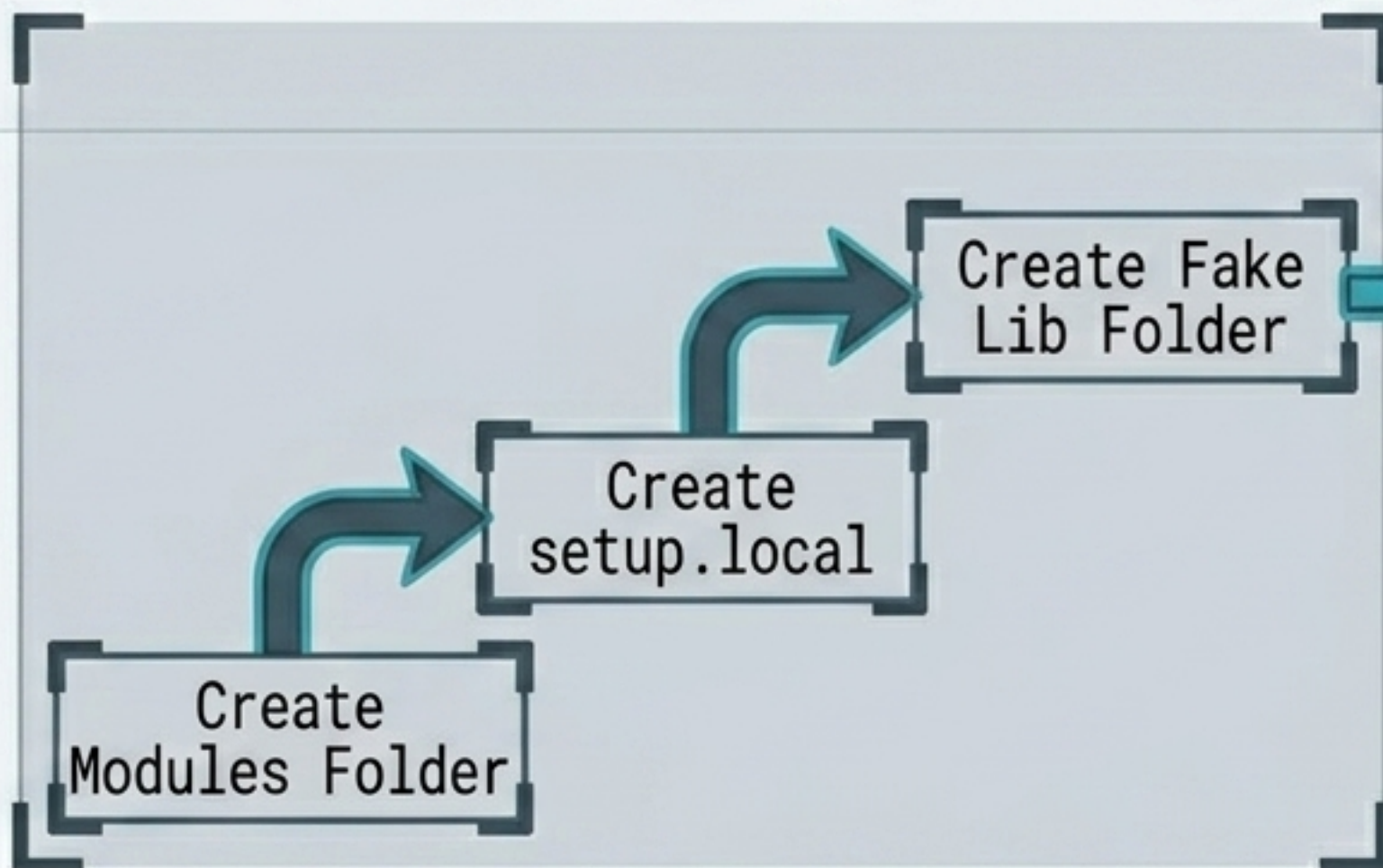




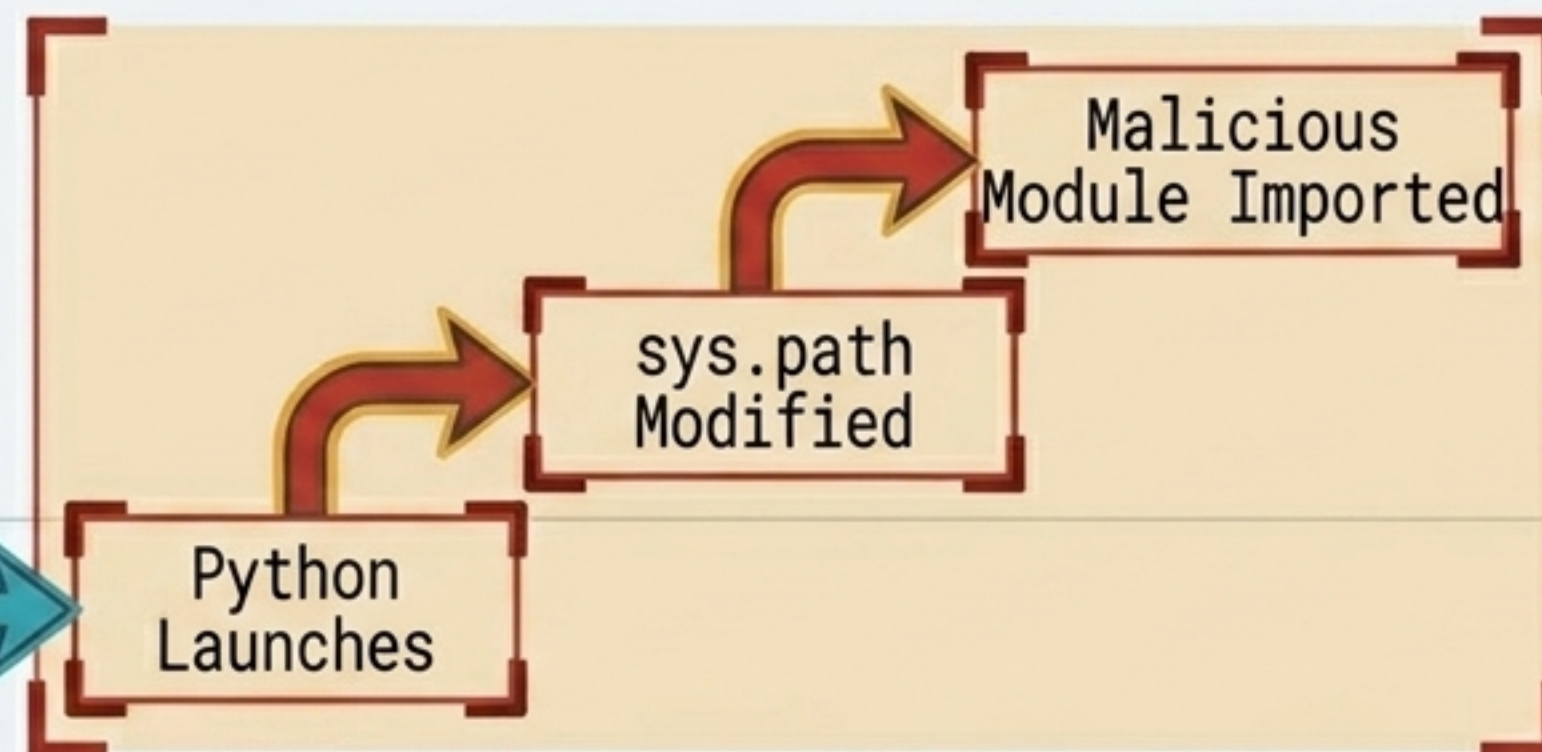
# The Exploit Kill-Chain

## Phase 1: Preparation (Low Priv User)

## Phase 1: Preparation (Low Priv User)



## Phase 2: Execution (Triggered Event)





## Attacker Prerequisites



System  
Requirement  
Local Access  
Required



Identity  
Requirement  
Low Privilege  
Account



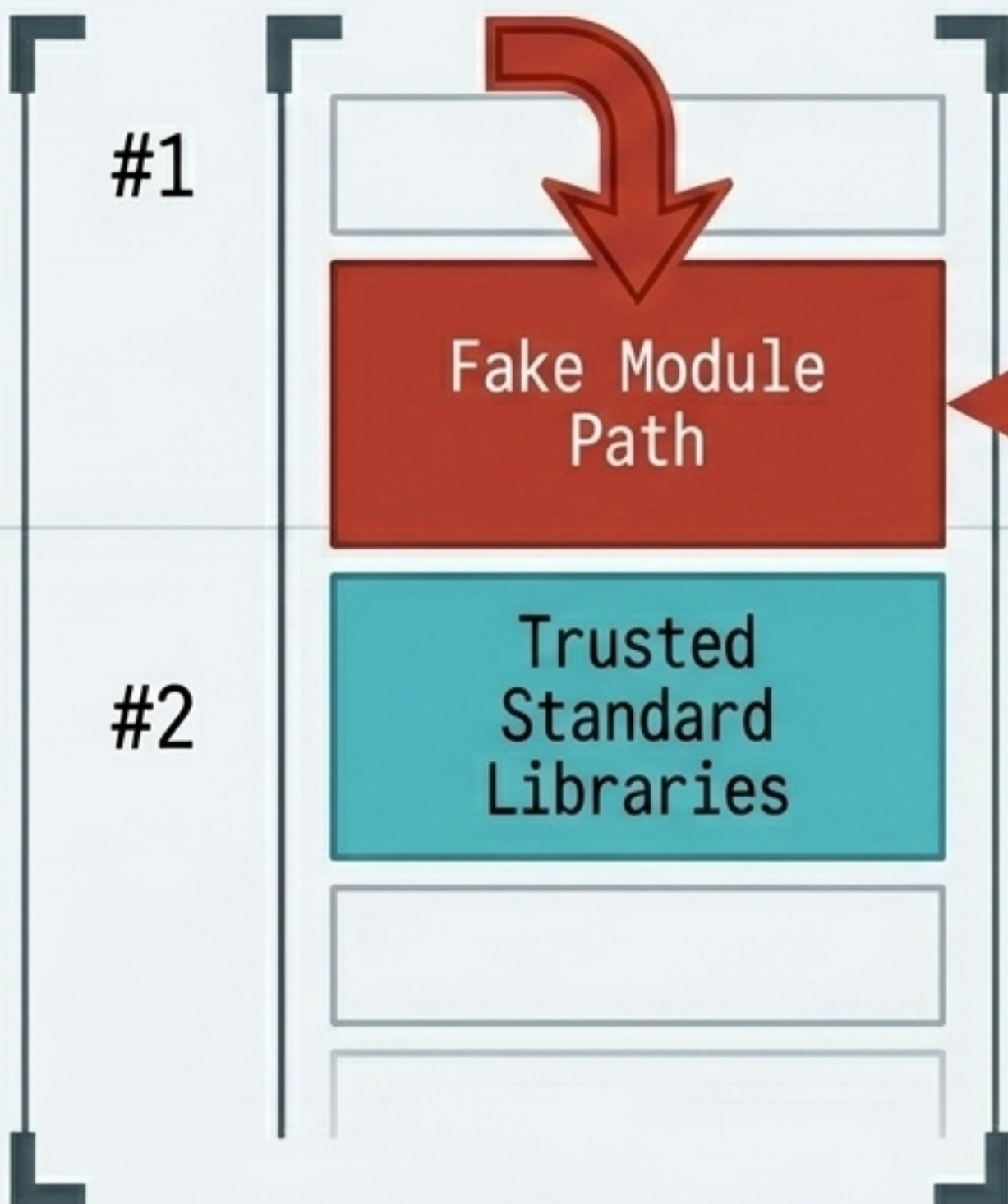
Environment  
Requirement  
Legacy Python  
Installation



User Action  
Requirement  
Python Application  
Execution



## Mechanism Deep-Dive: Poisoning sys.path



### CWE Classification: Import Order Manipulation

Python imports modules sequentially based on `sys.path`. When an attacker controls the search order, their fake module is imported first, resulting in immediate code execution before safe libraries are ever checked.

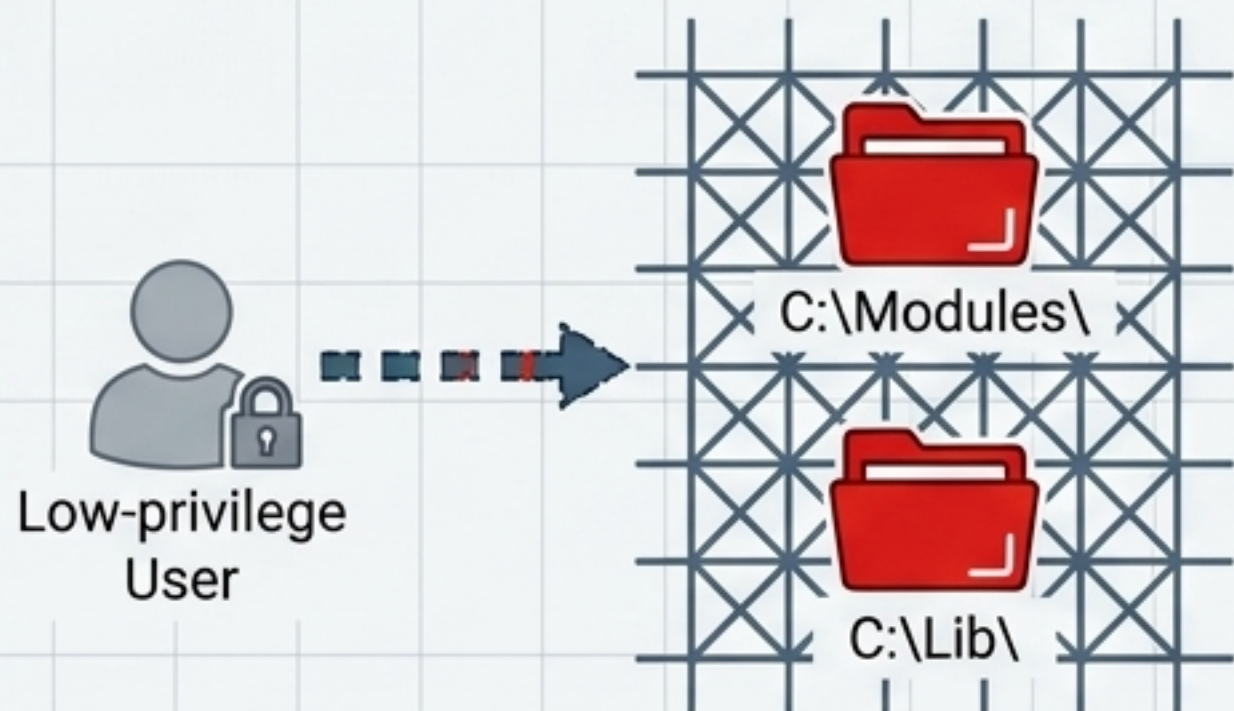
`import os`



# The Privilege Privilege Escalation Scenario

## Actor Interaction Map

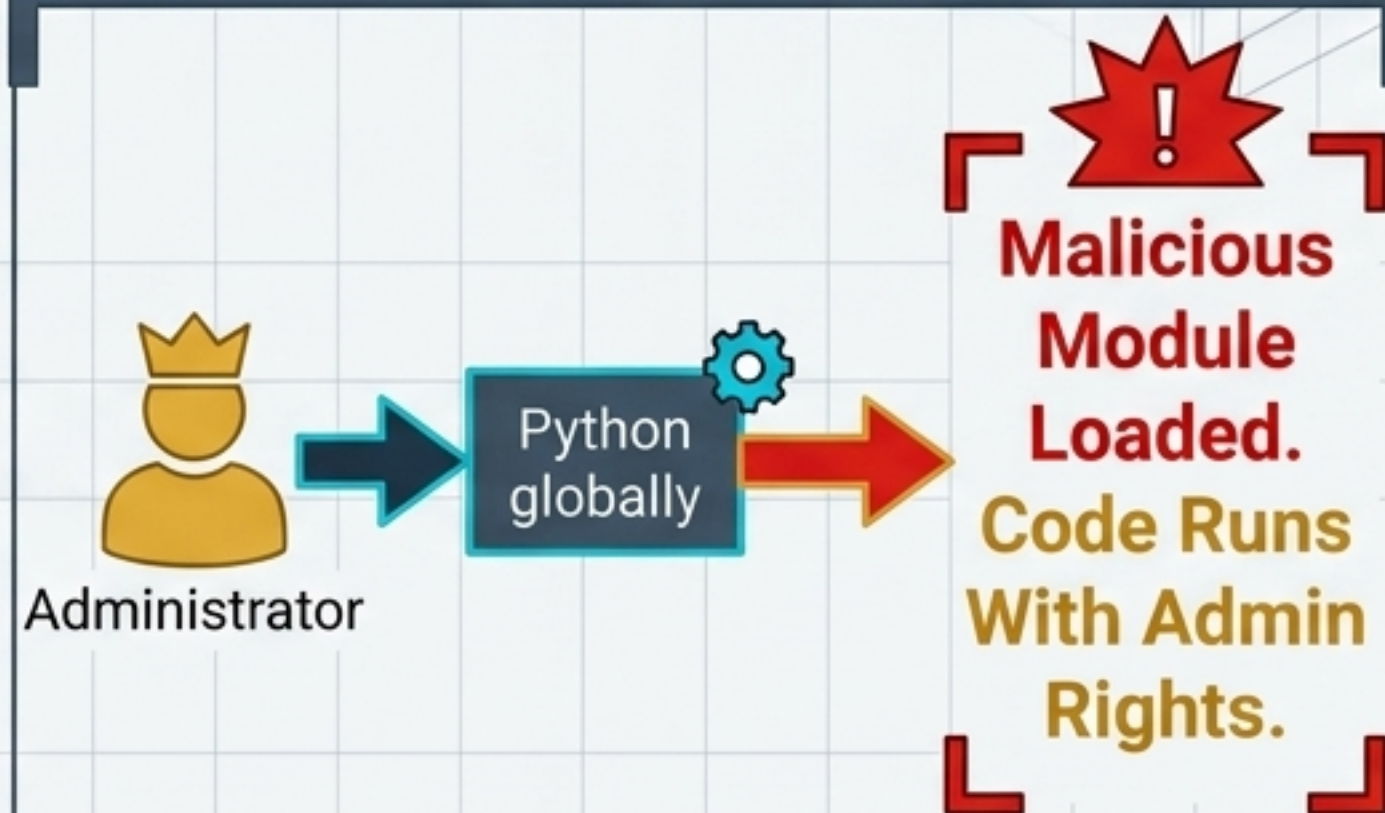
### Event A (The Setup)



### Time-Skip



### Event B (The Trigger)





# CVSS 4.0 Impact Dashboard



High (Read Sensitive Files)



High (Modify Python Scripts)



None (No direct DoS reported)

CVSS 4.0 SCORE

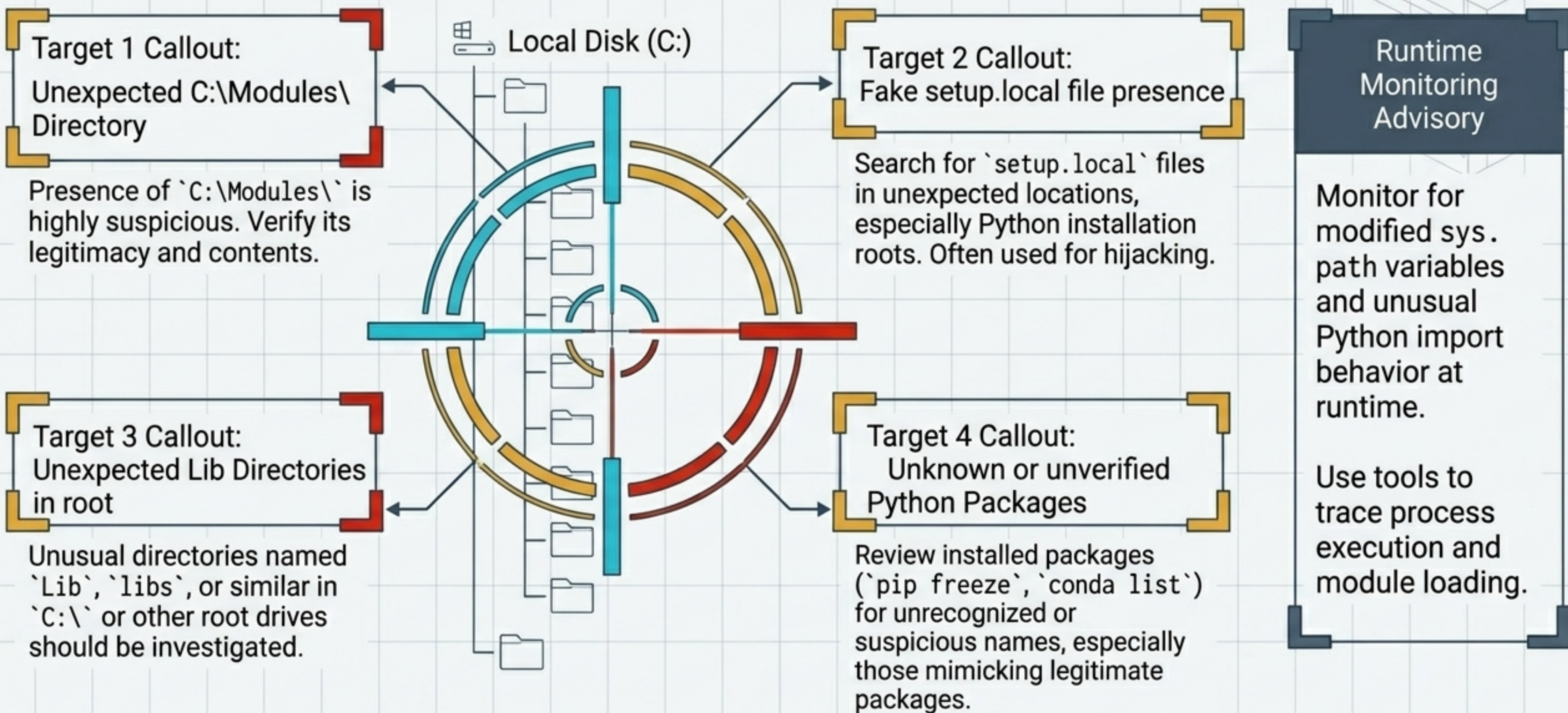
**5.3 Medium**

```
sysadmin@server:~$
```

```
CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:A/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N
```



# Indicators of Compromise (IoC Hunting)





# Mitigation & Remediation Controls



## Use Python Install Manager

Deprecate and replace all  
legacy installers.



Restrict user access to  
parent directories.



Create heavily protected  
system Modules directories.



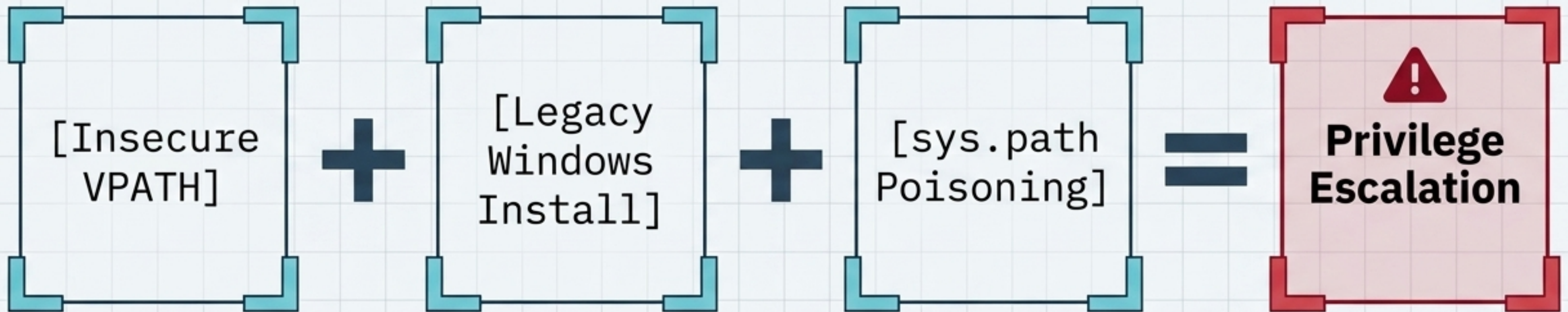
Actively monitor `sys.path`  
changes.



Upgrade to patched `Python`  
versions and remove all  
legacy installations.



# Synthesis: The Anatomy of CVE-2026-12003



By manipulating insecure landmark detection in legacy environments, attackers can arbitrarily control the module search path, silently elevating local privileges upon the next administrative execution.